The Doppler Effect: Using Sound as an Analogy

PHY4000_350: Black Holes, Big Bangs, & The Cosmos

Kyle Sharpe

2026-08-07

Table of Contents

Background....3
Demonstration Setup....5
  Hardware....5
  Software....5
  Procedure....6
Results....7
  Static Tests: Verifying System Functionality....7
Troubleshooting....9
  Overview of Attempts to Achieve Clean Frequency Data....9
  Hardware Limitations....9
  Software Attempts....10
  Root Cause Analysis....10
  Recommended Hardware Solutions & Alternative Approaches....11
  Lessons Learned....11
Conclusion....11
References....12

`

Disclaimer: Portions of this project, including troubleshooting, code modifications, and resource queries, were conducted with the assistance of Mistral Vibe, an AI tool. All AI-generated content has been reviewed, validated, and integrated responsibly in accordance with academic integrity guidelines.

Background

The Doppler effect is the change in wavelength/frequency caused by relative motion between source and observer [1]. In cosmology, this principle explains redshift—the lengthening of light wavelengths from distant galaxies—providing direct evidence for the universe's expansion [2]. Sound waves offer a tangible analogy for this effect. The Doppler effect in sound waves provides a direct, hands-on analogy for cosmological redshift: just as a moving sound source shifts the observed frequency of sound waves, the motion of galaxies shifts the wavelength of light (redshift/blueshift) [1][2]. While sound waves require a medium and propagate at non-relativistic speeds, the underlying principle—frequency shift due to relative motion—is universal [1].

This project uses sound to demonstrate the Doppler effect, establishing a practical foundation for understanding how relative motion affects wave frequency, a concept that directly parallels cosmological redshift. Unlike electromagnetic waves, sound is a longitudinal wave (compression/rarefaction) and cannot model the expansion of spacetime or relativistic effects that define cosmological redshift [1]. Additionally, cosmological redshift arises from the expansion of spacetime itself, not motion through a medium, and relativistic effects such as time dilation are negligible in sound but critical for light at high velocities [2]. Despite these limitations, the sound-based analogy effectively bridges the gap between everyday physics and cosmological phenomena.

The Doppler effect for sound is governed by the equation:

Equation 1: observed sound frequency

$$f' = f \frac{v \pm v_o}{v \mp v_s}$$

where:

- f' is the observed frequency
- f is the emitted frequency
- v is the speed of sound in air
- v_o is the observer's velocity (positive if moving toward the source)
- v_s is the source's velocity (positive if moving toward the observer) [1]

This equation mirrors the relativistic Doppler formula for light:

Equation 2: observed light wavelength (motion along line of sight)

$$\lambda' = \lambda \sqrt{\frac{1+\beta}{1-\beta}}$$

where:

- λ' is the observed wavelength
- λ is the emitted wavelength
- β is the velocity ratio defined as $\beta = \frac{v}{c}$ where:
  - v is the relative velocity between the source and observer
  - c is the speed of light in a vacuum (approximately 299,792,458 m/s)

Both describe how relative motion alters the perceived frequency, though the relativistic formula accounts for time dilation at speeds approaching the speed of light. [3]

Returning to the sound analogy, if the observer is assumed to be stationary, $v_o = 0$ which simplifies Equation 1 to:

Equation 3: observed sound frequency (stationary observer)

$$f' = f \frac{v}{v \pm v_s}$$

The speed of sound depends on temperature, thus:

Equation 4: observed frequency of sound (moving observer & source)

$$v = 331 + 0.6\,T$$

where T is the temperature in Celsius. This demonstration assumes that the environmental temperature is held at a constant 20°C, which gives a speed of sound of $v \approx 343 \frac{m}{s}$. [4]

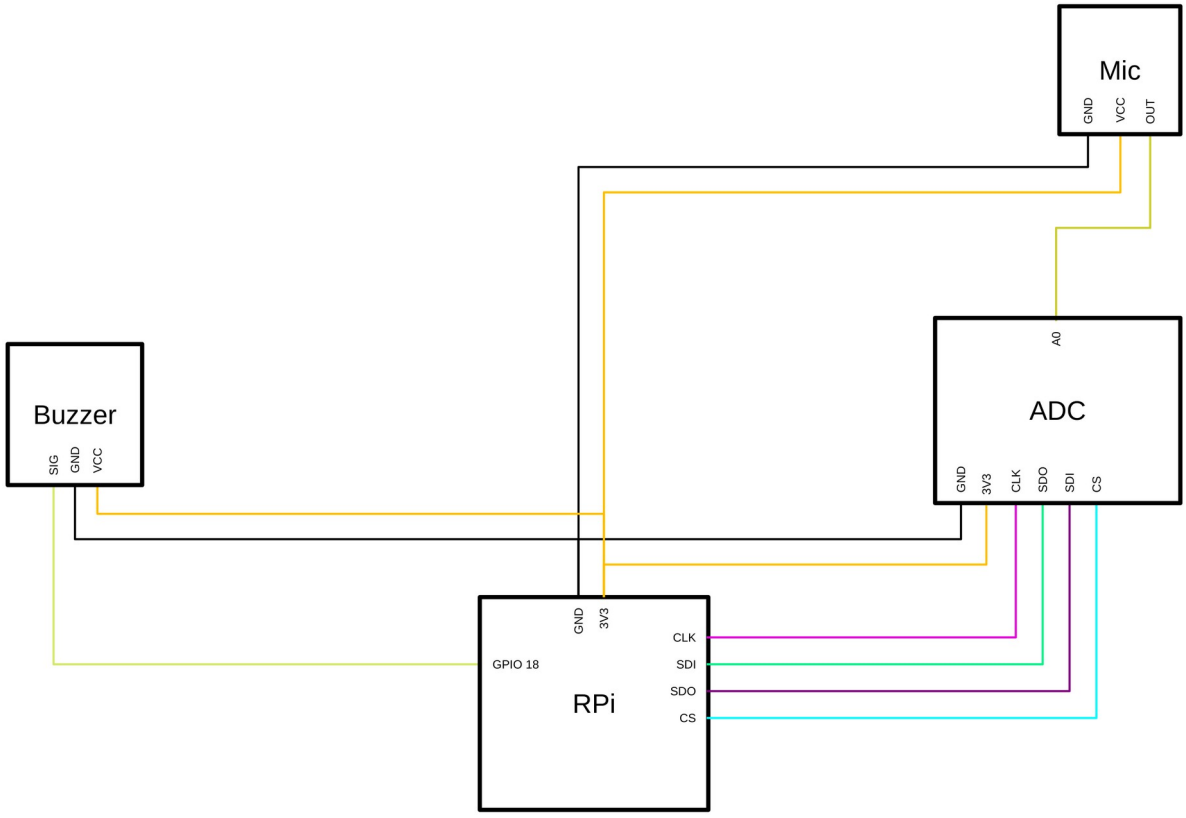
Demonstration Setup

Hardware

The system consists of a Raspberry Pi 5, a 12-bit MCP3208 ADC, a MAX4466 microphone amplifier, and a passive buzzer module (KC-1201) from a HiPi Sensor Kit 4.0 [5][6][7][8]. The Raspberry Pi 5 replaces the originally proposed Arduino, providing greater processing power and the capability to run Python for faster development and troubleshooting.

The wiring setup is as follows:

Figure 1: Wiring Diagram



Software

When sampling data, measurements must respect the Nyquist Theorem.

Equation 4: Nyquist Theorem

$$f_{sampling} > 2 f_{source}$$

This means that the source frequency must be less than half of the sampling frequency [9]. At a supply voltage of 2.7V (3.3V in this case from the pi), the ADC has a maximum sampling rate of 50,000 Hz, so the source frequency must be less than 25,000 Hz [7].

The buzzer is controlled by a PWM signal and can produce a wide range of audible frequencies, but is rated for 2,800 Hz at which it produces the cleanest sound wave [8]. Since this is less than half of the sampling frequency, the above condition is satisfied [9].

Software functionality is as follows. The MAX4466 captures the sound, which is digitized by the MCP3208 via SPI. The Raspberry Pi reads the digital data, applies a Fast Fourier Transform (FFT) for frequency detection, and logs results for analysis [10]. Multithreading enables concurrent buzzer control, ADC sampling, and frequency detection.

Procedure

The general procedure is as follows:

1. Position buzzer facing microphone
2. Start program
3. Begin data collection
4. Keep buzzer still if static test or move relative to microphone
5. Stop data collection and save trial data
6. Analyze data

Since time and material availability constraints limited the scope and final product, hand motions were chosen to move the buzzer for dynamic tests. To determine if this was feasible, calculations were performed to determine the resulting frequency shift from reasonably achievable motions.

For a 2800 Hz buzzer moving at 1 m/s:

- Approaching: $f' \approx 2800 \frac{343}{343-1} = 2808.19\,Hz$
- Receding: $f' \approx 2800 \frac{343}{343+1} = 2791.86\,Hz$
- Expected shift: $\pm 8.1\,Hz$

For a 2800 Hz buzzer moving at 0.5 m/s:

- Approaching: $f' \approx 2800 \frac{343}{343-0.5} = 2804.09\,Hz$
- Receding: $f' \approx 2800 \frac{343}{343+0.5} = 2795.92\,Hz$
- Expected shift: $\pm 4.08\,Hz$ [1]

The frequency shifts are very small and may not be noticeable if the data is noisy, but efforts have been made to filter and de-noise in software.

Results

Static Tests: Verifying System Functionality

Figure 2: Smoothing attempt causing undesired edge behavior

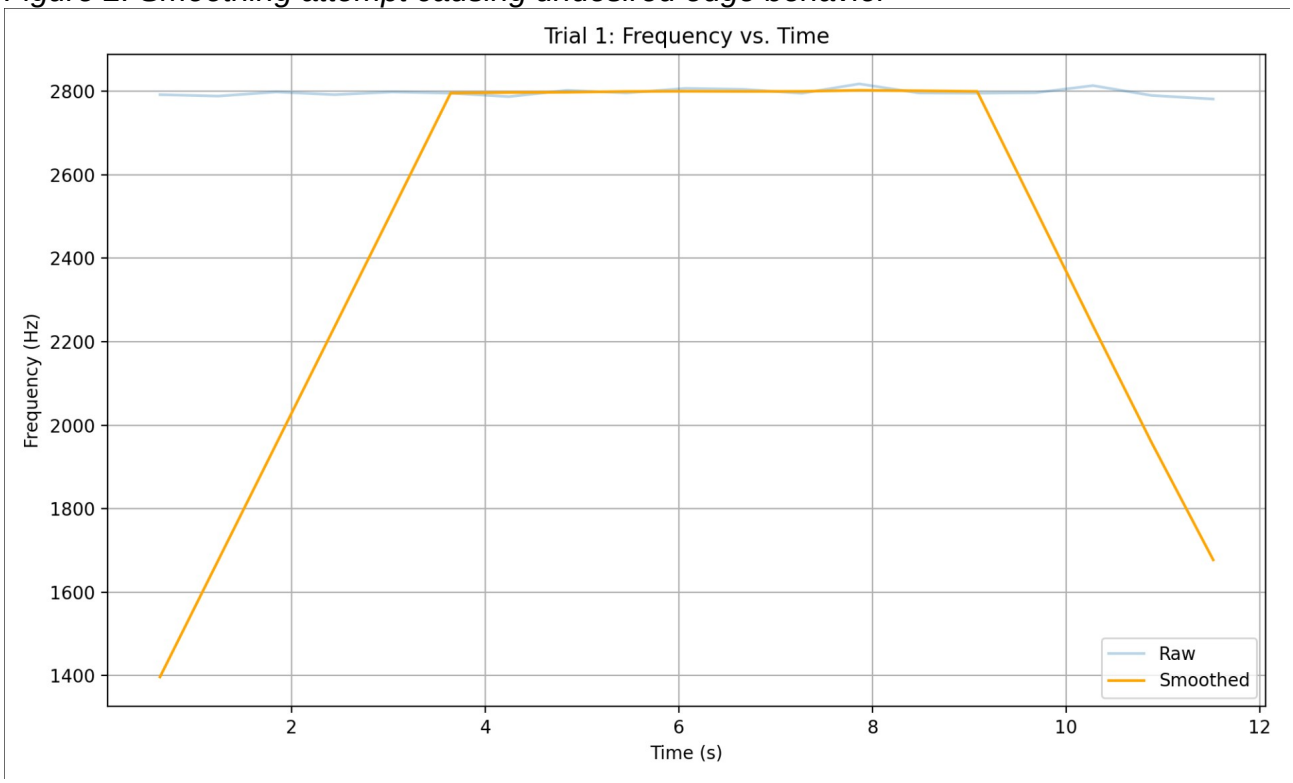


Figure 3: Data still susceptible to noise

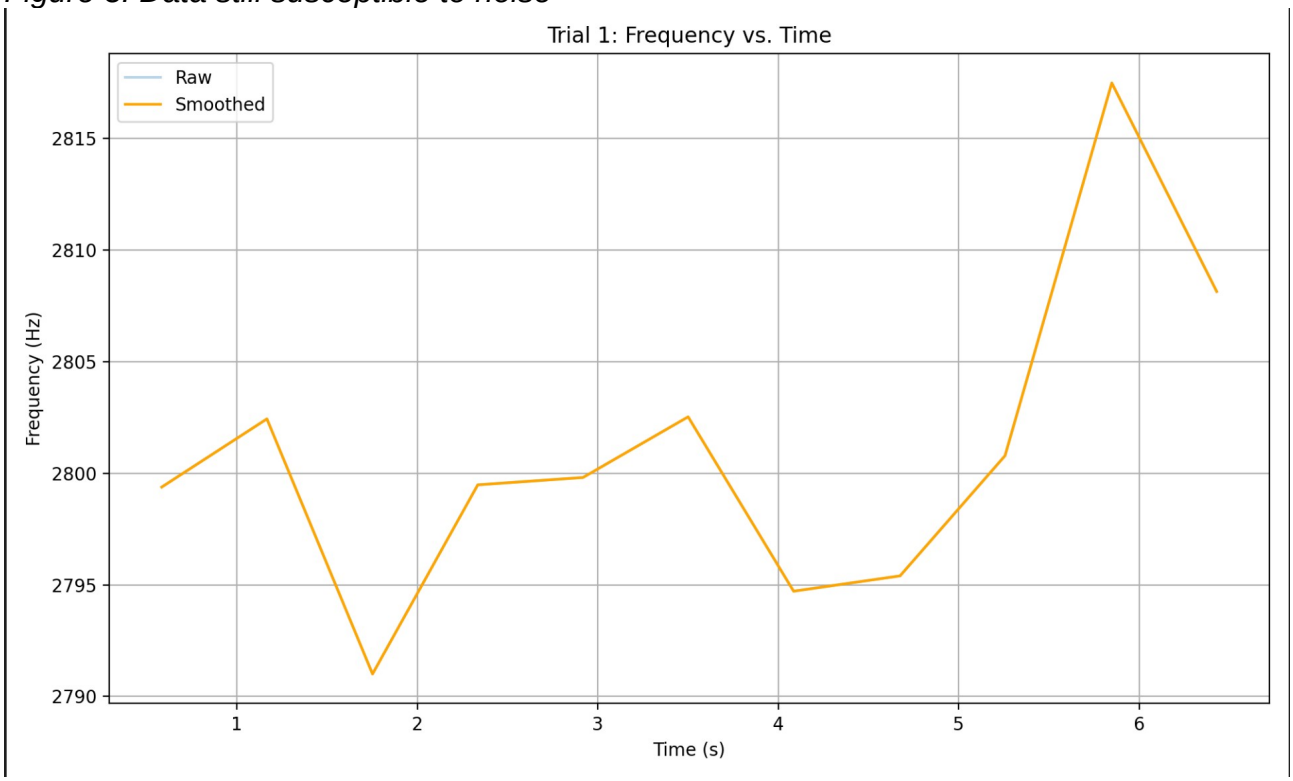


Figure 4: Attempt to reduce buffer size

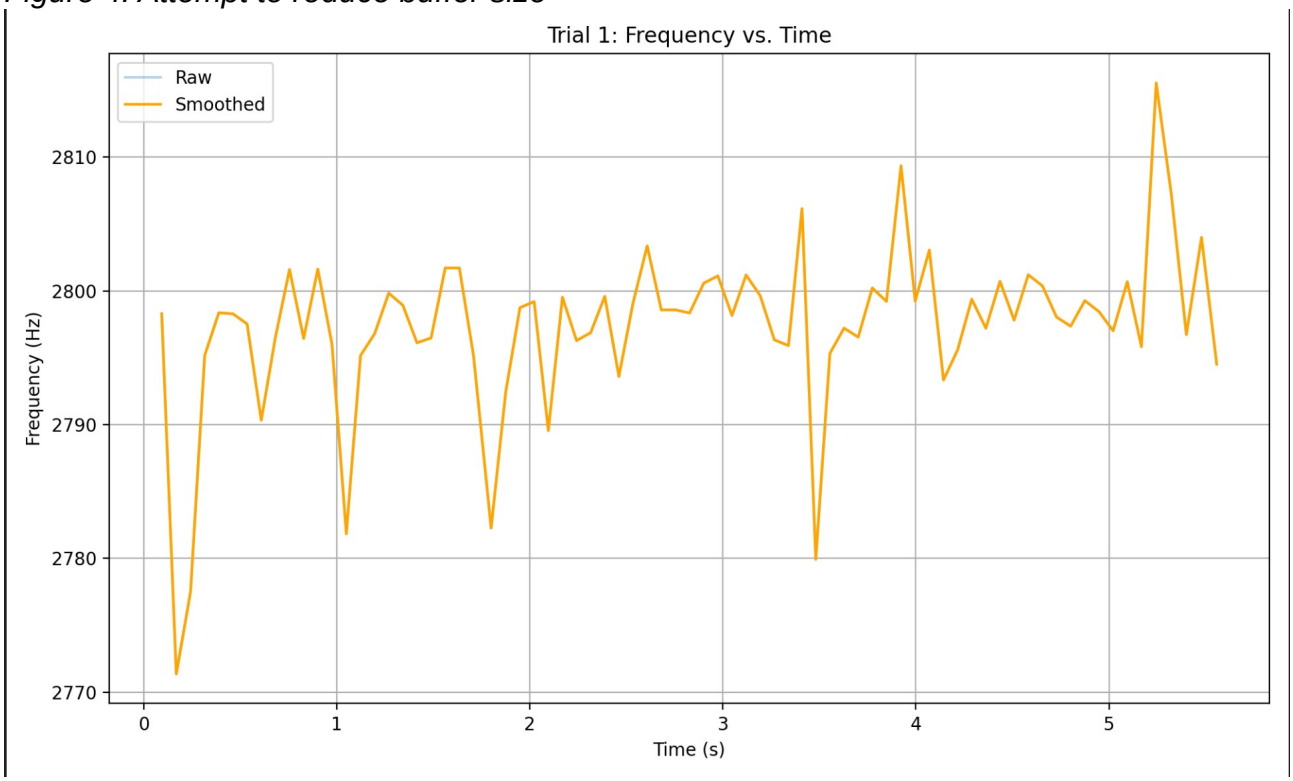


Figure 5: Narrowed search band to 10Hz

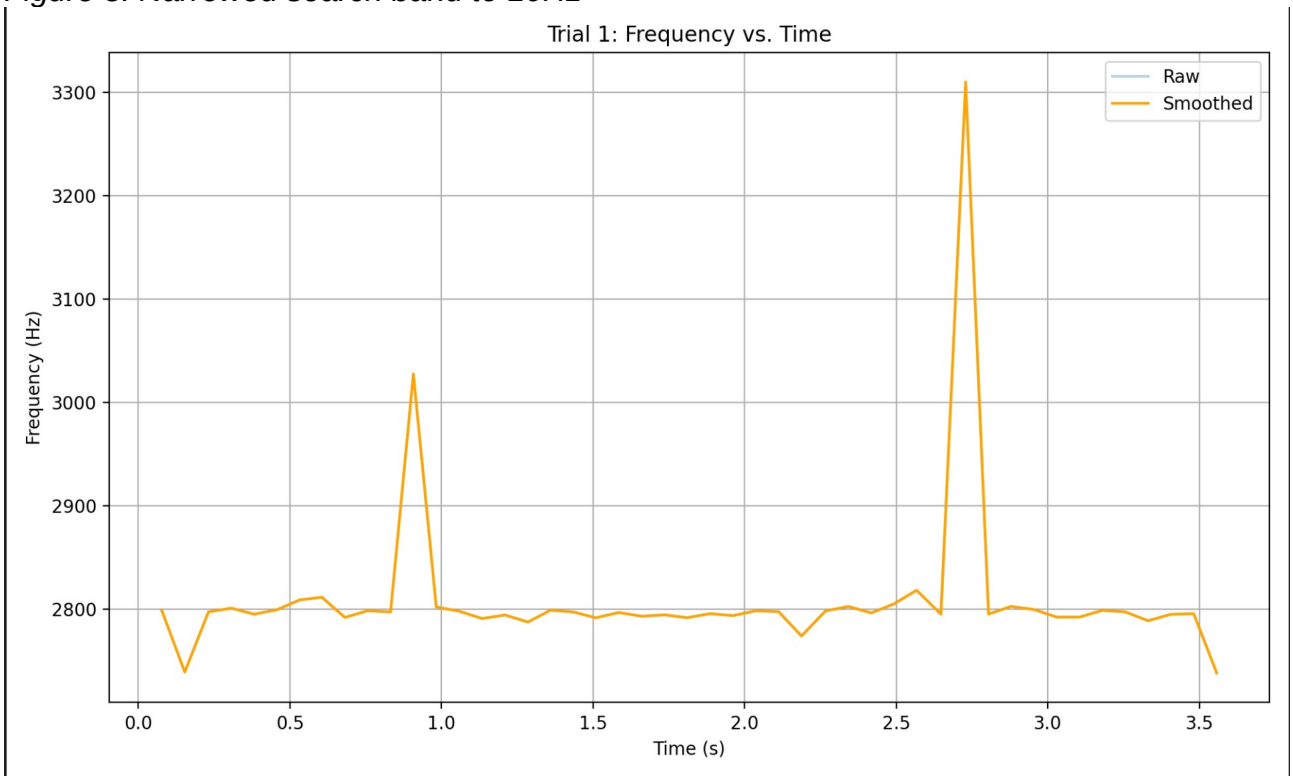
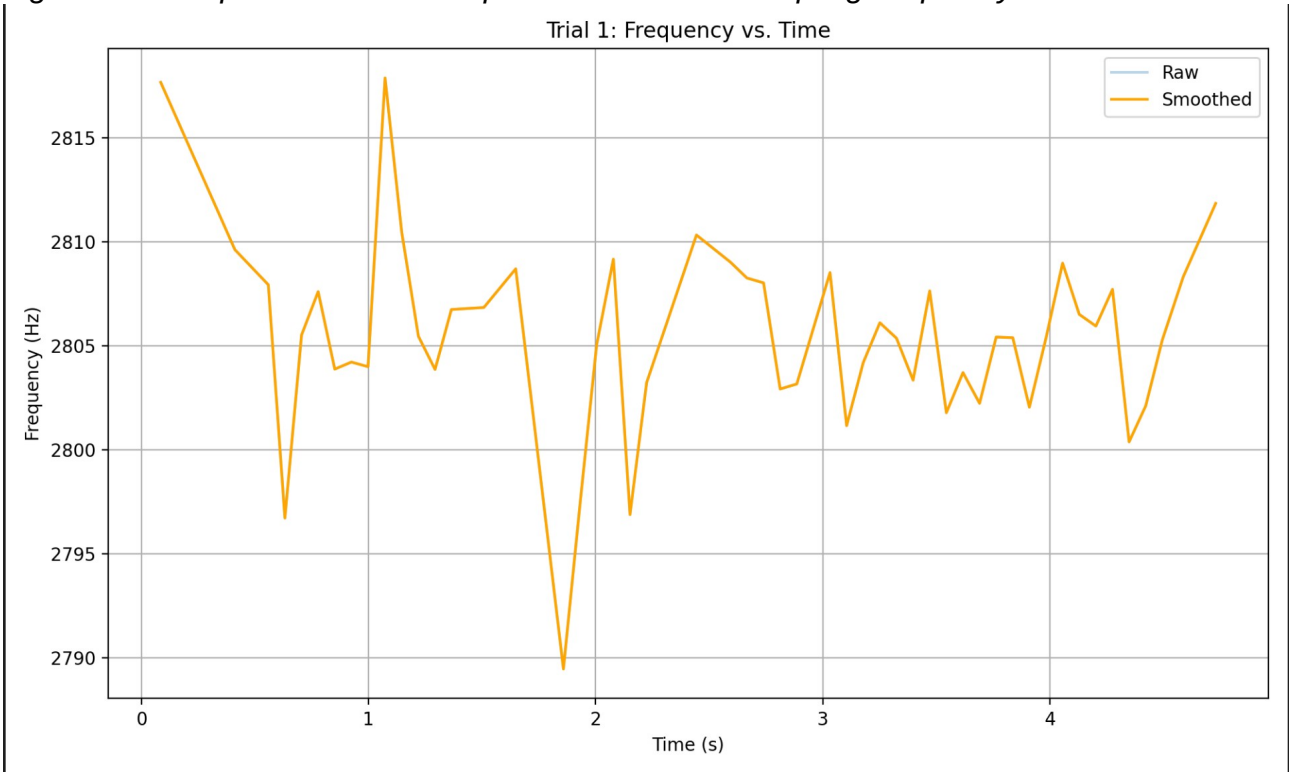


Figure 6: Attempt to reduce noise peaks and match sampling frequency to measured



Troubleshooting

Overview of Attempts to Achieve Clean Frequency Data

The primary challenge in this project was achieving sufficiently clean frequency data to demonstrate the Doppler effect using a Raspberry Pi 5, MCP3208 ADC, MAX4466 microphone amplifier, and a passive buzzer [5][6][7][8]. Despite multiple software-based attempts, the signal-to-noise ratio (SNR) remained too low to reliably detect the expected ±4–8 Hz frequency shifts caused by moving the buzzer at 0.5–1 m/s relative to the microphone [1]. The root cause was identified as a hardware limitation: the 12-bit resolution of the MCP3208 ADC, combined with the lack of analog low-pass filtering between the MAX4466 and ADC, introduced excessive noise that software alone could not eliminate [6][7].

Hardware Limitations

The MCP3208 provides only 12-bit resolution (4096 discrete levels over a 3.3V range), limiting the precision of amplitude measurements [7]. The MAX4466 microphone amplifier has a bandwidth of 100 kHz and a gain of 100x, which amplifies not only the 2800 Hz buzzer tone but also environmental and electrical noise across a wide frequency range [6] [8]. Without an analog low-pass filter (simple RC circuit) between the MAX4466 and MCP3208, high-frequency noise aliases into the sampled signal, corrupting the frequency domain analysis [11].

The initial target sampling rate of 44.1 kHz could not be achieved due to the overhead of reading samples one-by-one in a Python loop via SPI. The actual sampling rate varied

between 25–28 kHz, introducing jitter and inconsistencies in the FFT-based frequency estimation. The discrepancy between the assumed and actual sampling rates led to incorrect frequency scaling in the FFT, further degrading the accuracy of frequency estimates [10][12].

Software Attempts

Multiple software-based techniques were employed to improve frequency estimation. A Hanning window and FFT were used to estimate the dominant frequency in each buffer of samples. With a buffer size of 16384 and an actual sampling rate of ~27 kHz, the FFT bin width was ~1.65 Hz, theoretically sufficient, but noise caused the peak to jump between bins, resulting in ±15–20 Hz variations even for a static buzzer [14][15]. Zero-padding the buffer to 16384 samples did not significantly improve stability, as the noise floor remained too high [14].

Peak interpolation was implemented to estimate sub-bin frequency values, but noise created multiple local maxima, leading to incorrect interpolation and only marginal improvement with ±10–15 Hz variation [16]. The noisereduce library (spectral gating) was applied to the time-domain signal before FFT, but it was ineffective for a pure tone in high noise, and tuning parameters proved difficult [17]. A zero-phase moving average filter (filtfilt) with a window size of 100 reduced high-frequency jitter but could not compensate for underlying noise, leaving ±15 Hz spikes [18].

Further attempts included restricting the FFT search to ±10 Hz around the expected 2800 Hz tone and rejecting outliers >15 Hz from the expected frequency. However, outliers remained frequent, and the search band had to be wide enough to capture the Doppler shift, allowing noise through. Reducing the buffer size to 2048 or 512 to improve temporal resolution worsened the frequency estimation noise, as the FFT resolution decreased (bin width increased to ~13 Hz for 2048 samples at 27 kHz). Increasing the smoothing window to 200 created a trade-off between temporal and frequency resolution, with neither being sufficient [14].

Root Cause Analysis

The fundamental issue was the lack of analog filtering between the MAX4466 and MCP3208 [11]. The MAX4466 amplifies all frequencies up to 100 kHz, and without a low-pass filter, high-frequency noise (e.g., from the Raspberry Pi’s power supply or digital signals) aliases into the ADC’s sampling band. The 12-bit resolution of the MCP3208 further limits the dynamic range, making it difficult to distinguish the 2800 Hz tone from noise [7]. Additionally, the Python-based sample collection loop introduced timing jitter, as each SPI transaction to read a sample from the MCP3208 incurred variable overhead. This resulted in an inconsistent and lower-than-expected sampling rate, which directly affected the FFT’s frequency scaling [12].

Recommended Hardware Solutions & Alternative Approaches

To achieve the required precision for this project, several hardware modifications are recommended. A simple RC low-pass filter (e.g., 1 kΩ resistor + 1 µF capacitor) should be placed between the MAX4466 output and MCP3208 input to attenuate high-frequency noise above ~10 kHz [11]. Replacing the MCP3208 with a higher-resolution ADC (16-bit such as ADS1115) would improve the signal-to-noise ratio [18]. Implementing Direct Memory Access (DMA) on the Raspberry Pi to read ADC samples at a consistent rate without Python loop overhead is another option, requiring advanced GPIO configuration or a hardware SPI controller [19].

However, since hardware modifications are not feasible with the current time/cost constraints to order more components, alternative approaches include using simulated data to illustrate the concept, or using a smartphone app (e.g., Frequency Analyzer or Spectroid) to capture the Doppler shift [20].

Lessons Learned

The project highlighted several key lessons. Hardware limitations cannot be fully compensated by software in signal processing applications. Analog filtering (low-pass, band-pass) is critical for precise frequency measurements to remove out-of-band noise before digitization [11]. Accurate frequency analysis requires a consistent and known sampling rate, as variable rates introduce errors that propagate through the entire analysis pipeline [12]. Libraries like noisereduce, optimized for speech and audio signals, may not be suitable for pure tone detection in high-noise environments, and specialized DSP techniques (PLL and/or autocorrelation) may be more effective [16][21][22].

Conclusion

While the theoretical understanding of the Doppler effect and its analogy to cosmological redshift was simple to model and predict results for, the practical implementation was limited by hardware, time, and cost constraints. The project demonstrated that even with affordable components, the core principles of frequency shift due to relative motion can be explored, reinforcing the educational value of the sound-based analogy.

Key lessons learned include the importance of hardware precision, the critical role of analog filtering in signal processing, and the challenges of achieving consistent sampling rates for accurate frequency analysis. The analogy effectively bridges the gap between everyday physics and cosmological phenomena, though it is important to acknowledge its limitations: sound waves cannot model the expansion of spacetime or relativistic effects that define cosmological redshift. Future work should focus on improving the SNR through analog filtering or higher-resolution ADCs, or pivot to a simulated demonstration if hardware modifications are not possible.

References

[1]	OpenStax, “17.8: The Doppler Effect,” Nov. 2016. Accessed: Aug. 07, 2026. [Online]. Available: https://phys.libretexts.org/Bookshelves/University_Physics/University_Physics_(OpenStax)/Book:_University_Physics_I_-_Mechanics_Sound_Oscillations_and_Waves_(OpenStax)/17:_Sound/17.08:_The_Doppler_Effect
[2]	L. Hustak, “Hubble Cosmological Redshift - NASA Science,” Sep. 2024. Accessed: Aug. 07, 2026. [Online]. Available: https://science.nasa.gov/mission/hubble/science/science-behind-the-discoveries/hubble-cosmological-redshift/
[3]	OpenStax, “5.8: Doppler Effect for Light,” Nov. 2016. Accessed: Aug. 07, 2026. [Online]. Available: https://phys.libretexts.org/Bookshelves/University_Physics/University_Physics_(OpenStax)/University_Physics_III_-_Optics_and_Modern_Physics_(OpenStax)/05:__Relativity/5.08:_Doppler_Effect_for_Light
[4]	Wikipedia Contributors, “Speed of Sound,” Wikimedia Foundation, May 2019. Accessed: Aug. 07, 2026. [Online]. Available: https://en.wikipedia.org/wiki/Speed_of_sound
[5]	Raspberry Pi Ltd, “Raspberry Pi 5.” Raspberry Pi Ltd, Oct. 2023. [Online]. Available: https://www.lcsc.com/datasheet/C20625699.pdf
[6]	Analog Devices, “Low-Cost, Micropower, SC70/SOT23-8, Microphone Preamplifiers with Complete Shutdown.” Analog Devices, Jun. 2012. [Online]. Available: https://www.analog.com/media/en/technical-documentation/data-sheets/MAX4465-MAX4469.pdf
[7]	Microchip, “2.7V 4-Channel/8-Channel 12-Bit A/D Converters with SPI Serial Interface.” Microchip, Jan. 02, 2008. [Online]. Available: https://ww1.microchip.com/downloads/aemDocuments/documents/APID/ProductDocuments/DataSheets/21298e.pdf
[8]	Kingstate, “Magnetic Transducer.” Kingstate. [Online]. Available: https://datasheet4u.com/download/703459/KC-1201.html
[9]	Wikipedia Contributors, “Nyquist–Shannon Sampling Theorem,” Wikimedia Foundation, Nov. 2019. [Online]. Available: https://en.wikipedia.org/wiki/Nyquist%E2%80%93Shannon_sampling_theorem
[10]	Wikipedia Contributors, “Fast Fourier Transform,” Wikimedia Foundation, Aug. 2019. [Online]. Available: https://en.wikipedia.org/wiki/Fast_Fourier_transform
[11]	Wikipedia Contributors, “Low-Pass Filter,” Wikimedia Foundation, Jul. 2019. [Online]. Available: https://en.wikipedia.org/wiki/Low-pass_filter
[12]	T. M. Souders, D. R. Flach, C. Hagwood, and G. L. Yang, “The Effects of Timing Jitter in Sampling Systems,” *IEEE Transactions on Instrumentation and Measurement*, vol. 39, no. 1, pp. 80–85, 1990, doi: 10.1109/19.50421.
[13]	National Instruments, “Understanding FFTs and Windowing Overview,” [Online]. Available: https://download.ni.com/evaluation/pxi/Understanding%20FFTs%20and%20Windowing.pdf
[14]	Wikipedia Contributors, “Window Function,” Dec. 2020. [Online]. Available:

	https://en.wikipedia.org/wiki/Window_function
[15]	J. O. Smith, “Interpolating a DFT,” Stanford.edu. [Online]. Available: https://ccrma.stanford.edu/~jos/sasp/Interpolating_DFT.html
[16]	T. Sainburg, “Noisereduce: Noise Reduction Using Spectral Gating in Python,” Oct. 2024. [Online]. Available: https://pypi.org/project/noisereduce/
[17]	SciPy, “Scipy.Signal.Filtfilt — SciPy V1.8.0 Manual,” Jun. 2026. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.filtfilt.html
[18]	Texas Instruments, “ADS111x Ultra-Small, Low-Power, I 2 C-Compatible, 860SPS, 16-Bit ADCs With Internal Reference, Oscillator, and Programmable Comparator,” Dec. 2024. [Online]. Available: https://www.ti.com/lit/ds/symlink/ads1115.pdf
[19]	fandahao17, “GitHub - fandahao17/Raspberry-Pi-DMA-Tutorial: A Step-by-Step Guide on Using DMA to Achieve High-Speed Sampling on Raspberry Pi,” GitHub. [Online]. Available: https://github.com/fandahao17/Raspberry-Pi-DMA-Tutorial
[20]	Rikki Systems Inc, “Sonic Tools SVM,” AppRecs.com. [Online]. Available: https://apprecs.com/ios/1245046029/sonic-tools-svm
[21]	Y.-Q. Tu and Y.-L. Shen, “Phase Correction Autocorrelation-Based Frequency Estimation Method for Sinusoidal Signal,” *Signal Processing*, vol. 130, pp. 183–189, 2017, doi: https://doi.org/10.1016/j.sigpro.2016.06.012.
[22]	R. Sudhakar, R. Agarwal, and S. D. Roy, “Frequency Estimation Based on Iterated Autocorrelation Function,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 33, no. 1, pp. 70–76, 1985, doi: 10.1109/TASSP.1985.1164540.